DMR++

Table of Contents

1. Introduction . . . 1
2. How Does It Work? . . . 2
3. Supported Data Formats . . . 3
   3.1. hdf5 . . . 3
   3.2. *HDF4/HDF4-EOS2* . . . 5
4. Building DMR++ files for HDF4 and HDF4-EOS2 (experimental) . . . 5
   4.1. Using get_dmrpp_h4 . . . 5
5. Building DMR++ files for HDF5/NetCDF4 with *get_dmrpp* . . . 11
   5.1. Using *get_dmrpp* . . . 11
   5.2. Command line options . . . 11
   5.3. *hdf5_handler* Configuration . . . 14
   5.4. The *H5.EnableCF* option . . . 14
   5.5. Missing data, the CF conventions and *hdf5* . . . 15
6. Hyrax - Serving data using DMR++ files . . . 15
   6.1. Using DMR++ with http(s) URLs . . . 15
   6.2. Using DMR++ with file URLs . . . 16
   6.3. Using DMR++ with the template string (NASA). . . . 16
7. Recipe: Building and testing DMR++ files . . . 17
   7.1. Recipe: Building DMR++ files using a Hyrax docker container . . . 17
   7.2. Testing and qualifying DMR++ files . . . 19
   7.3. Flattening Groups . . . 19
   7.4. DAP representations . . . 19

1. Introduction

The DMR++ is a sidecar file that provides a fast and flexible way to serve data stored in S3. The DMR++ encodes the location of the data content residing in a binary data file/object (e.g., an hdf5 file) so that it can be directly accessed, without the need for an intermediate library API, by using the file with the location information. The binary data objects may be on a local filesystem, or they may reside across the web in something like an S3 bucket.

The DMR++ adds to the *DMR* of the datafile *extra* information, hence the ++ (for more on *DMR* see Sections 1.5.7-1.5.15), such as `byte offsets`, `chunk references`, `compression`, among other relevant variables that can be used to get cloud-performant access to HDF5/NetCDF4 datafiles on S3. Hyrax, the OPeNDAP data Server, uses DMR++ to efficiently subset data on a local filesytem or S3.

Additional advantages to the DMR++ are:

1. Relieves the burden from Data Producers to *transform and reformat* massive dataset collections, avoiding the need to store multiple copies of the same files.
2. All chunks inside the `h5` file remain inside the file, i.e. each file retains its integrity. The location of the chunks inside the `h5` file are specified within the DMR++ .
3. A DMR++ can be easily and programatically generated by OPeNDAP's Hyrax Data Server for HDF5/NetCDF4 files and most of HDF4 files.

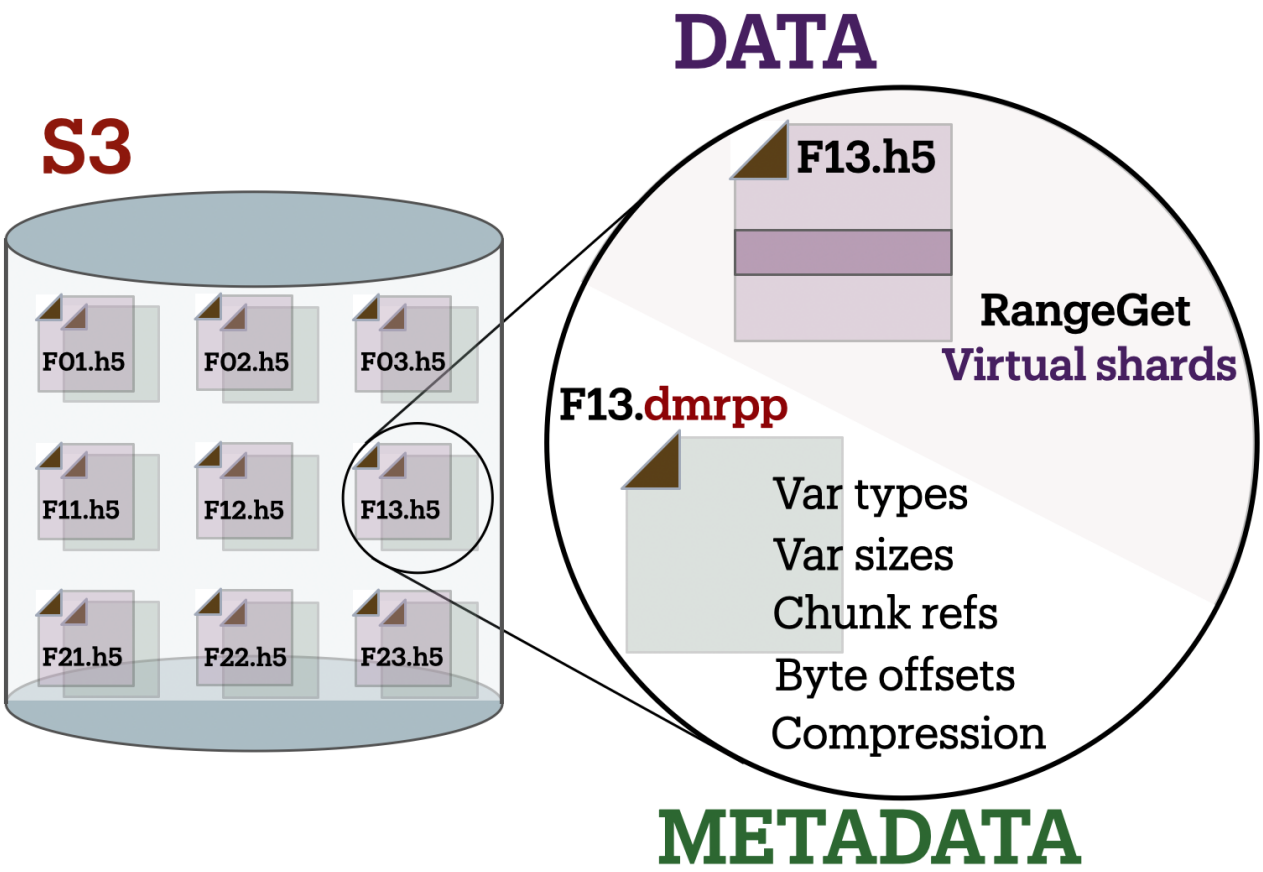

Figure 1. Diagram of a collection of HDF5 files on an S3 bucket. Each file has next to it a side car file with the same name as the original but with ".dmrpp" (i.e. the DMR++). The ".dmrpp" file need not be next to the source file, and in fact because it is much smaller than the ".h5" source file, it is often moved in and out of S3 by the Hyrax data server, to reduce latency within S3.

2. How Does It Work?

The DMR++ ingest software reads a data file (see this note) and builds a document that holds all of the file's metadata (the names and types of all of the variables along with any other information bound to those variables). This information is stored in a document we call the Dataset Metadata Response (DMR). The DMR++ adds some extra information to this (that's the ++ part) about where each variable can be found and how to decode those values. The DMR++ is simply a special annotated DMR document.

This effectively decouples the annotated DMR (DMR++) from the location of the granule file itself. Since DMR++ files are typically significantly smaller than the source data granules they represent, they can be stored and moved for less expense. They also enable reading all of the file's metadata in one operation instead of the iterative process that many APIs require.

If the DMR++ contains references to the source granules location on the web, the location of the the DMR++ file itself does not matter.

Software that understands the DMR++ content can directly access the data values held in the source granule file, and it can do so without having to retrieve the entire file and work on it locally, even when the file is stored in a Web Object Store like S3.

If the granule file contains multiple variables and only a subset of them are needed, the DMR++ enabled software can retrieve just the bytes associated with the desired variables parts.

NOTE	The OPeNDAP software currently supports HDF5 and NetCDF4. Other formats can be supported, such as zarr.

3. Supported Data Formats

The DMR++ software currently works with 'hdf5', 'netcdf-4', and (experimental as of 8/29/24) "HDF4"/"HDF4-EOS2" files. (The 'netcdf-4' format is a subset of "hdf5" so "hdf5" tools are utilized for both.) Other formats like "zarr", "netcdf-3" are not currently supported by the DMR++ software, but support could be added if requested. However, an external group working on the Python Kerchunk software has developed VirtualiZarr which can parse either Kerchunk or DMR++ documents and read from data those describe using the Zarr API.

3.1. hdf5

The 'hdf5' data format is quite complex and many of the options and edge cases are not currently supported by the DMR++ software.

These limitations and how to quickly evaluate an "hdf5" or "netcdf-4" file for use with the DMR++ software are explained below.

3.1.1. "hdf5" filters

The "hdf5" format has several filter/compression options used for storing data values. The DMR++ software currently supports data that utilize the H5Z_FILTER_DEFLATE, H5Z_FILTER_SHUFFLE, and H5Z_FILTER_FLETCHER32 filters. You can find more on hdf5 filters here.

3.1.2. "hdf5" storage layouts

The "hdf5" format also uses a number of "storage layouts" that describe various structural organizations of the data values associated with a variable in the granule file. The DMR++ software currently supports data that utilize the H5D_COMPACT, H5D_CHUNKED, and H5D_CONTIGUOUS storage layouts. These are all of the storage layouts defined by the "hdf5" library, but others can be added. You can find more on hdf5 storage layouts here.

3.1.3. Is my "hdf5" or "netcdf-4" file suitable for DMR++ ?

To determine the "hdf5" filters, storage layouts, and chunking scheme used in an "hdf5" or "netcdf-4" file you can use the command:

```
h5dump -H -p <filename>
```

To get a human readable assessment of the file that will show the storage layouts, chunking structure, and the filters needed for each variable (aka DATASET in the *hdf5* vocabulary) h5dump info can be found here.

h5dump example output:

```
$ h5dump -H -p chunked_gzipped_fourD.h5
HDF5 "chunked_gzipped_fourD.h5" {
GROUP "/" {
  DATASET "d_16_gzipped_chunks" {
     DATATYPE  H5T_IEEE_F32LE
     DATASPACE  SIMPLE { ( 40, 40, 40, 40 ) / ( 40, 40, 40, 40 ) }
     STORAGE_LAYOUT {
        CHUNKED ( 20, 20, 20, 20 )
        SIZE 2863311 (3.576:1 COMPRESSION)
     }
     FILTERS {
        COMPRESSION DEFLATE { LEVEL 6 }
     }
     FILLVALUE {
        FILL_TIME H5D_FILL_TIME_ALLOC
        VALUE  H5D_FILL_VALUE_DEFAULT
     }
     ALLOCATION_TIME {
        H5D_ALLOC_TIME_INCR
     }
  }
 }
}
```

3.1.4. Is my netcdf file *netcdf-3* or *netcdf-4*?

A file with the suffix *.nc4* is recognized as a *netcdf-4* file. However, the file suffix *.nc* can be the commonly used naming convention for both *netcdf-3* and *netcdf-4* files. You can use the command:

```
ncdump -k <filename>
```

to determine if a *netcdf* file is either classic *netcdf-3* (classic) or *netcdf-4* (netCDF-4). You can learn more in the NetCDF documentation here.

NOTE	The *netcdf* library must be installed on the system upon which the command is issued.

3.2. *HDF4/HDF4-EOS2*

This is a complicated case, and its support as of 8/29/24 is still considered experimental. The HDF4 data model is quite complex, more so than the HDF5 model, and we focusing on complete support for those features used by NASA. To this end, we are also working on support for HDF4-EOS2, data files that can only be read correctly with the HDF4-EOS2 library. The main distinction of that API is the treatment of values for the Domain variables for Latitude and Longitude. Our support handles the HDF4-EOS Grid data type and using DMR++ the Latitude and Longitude values appear as users expect, although some aspects of this are ongoing. We do not yet support the HDF4-EOS2 Swath data type.

Se the section below for information on the tool for building DMR++ files for HDF4 and HDF4-EOS2 data files.

4. Building DMR++ files for HDF4 and HDF4-EOS2 (experimental)

The HDF4 and HDF4-EOS2 (hereafter just HDF4) DMR++ document builder is currently available in the docker container we build for hyrax server/service. You can get this container from the public Docker Hub repository. You can also get and build the "Hyrax" source code, and use the client that way (as part of a source code build) but it's much more complex than getting the Docker container. In addition, the Docker container includes a server that can test the DMR++ documents that are built and can even show you how the files would look when served without using the DMR++ .

NOTE	The followingg commands should be consider still experimental and subject to some change. Modify it to suit your own needs.

4.1. Using get_dmrpp_h4

Make a new directory in a convenient place and copy the HDF4 and/or HDF4-EOS2 files in that directory. Once you have the files in that directory, make an environment variable so it can be referred to easily. From inside the directory:

```
export HDF4_DIR=$(pwd)
```

Get the Docker container from Docker Hub using this command:

```
docker run -d -h hyrax -p 8080:8080 -v $HDF4_DIR:/usr/share/hyrax --name=hyrax
opendap/hyrax:snapshot
```

What the options mean:

```
-d, --detach Run container in background and print container ID
-h, --hostname Container host name
-p, --publish Publish a container's port(s) to the host
-v, --volume Bind mount a volume
--name Assign a name to the container
```

This command will fetch the container **opendap/hyrax:snapshot** from Docker Hub. Thw *snapshot* is the latest build of the container. It will then *run* the container and return the container ID. The *hyrax* server is now running on you computer and can be accessed with a web browser, curl, et cetera. More on that in a bit.

The volume mount, from `$HDF4_DIR` to `'/usr/share/hyrax'` mounts the current directory of the host computer running the container to the directory */usr/share/hyrax* inside the container. That directory is the root of the server's data tree. This means that the HDF4 files you copied into the `HDF4_DIR` directory will be accessible by the server running in the container. That will be useful for testing later on.

Note: If you want to use a specific container version, just substitute the version info for *snapshot.*

Check that the container is running using:

```
docker ps
```

This will show a somewhat hard-to-read bit of information about all the running Docker container on you host:

```
CONTAINER ID        IMAGE                 COMMAND              CREATED          STATUS
PORTS                     NAMES
2949d4101df4   opendap/hyrax:snapshot   "/entrypoint.sh -"   15 seconds ago   Up 14
seconds   8009/tcp, 8443/tcp,
10022/tcp, 11002/tcp, 0.0.0.0:8080->8080/tcp   hyrax
```

If you want to stop the containers, use

```
docker rm -f <CONTAINER ID>
```

where the `<CONTAINER ID>` for the one we just started and shown in the output of *docker ps -a* above is *2949d4101df4*. No need to stop the container now, I'm just pointing out how to do it because it's often useful.

4.1.1. Lets run the DMR++ builder

NOTE At the end of this, I'll include a shell script that takes away many of these steps, but

> the script obscures some aspects of the command that you might want to tweak, so the following shows you all the details. Skip to **Simple shell command** to skip over these details.

Make sure you are in the directory with the HDF4 files for these steps.

Get the command to return its help information:

```
docker exec -it hyrax get_dmrpp_h4 -h
```

will return:

```
usage: get_dmrpp_h4 [-h] -i I [-c CONF] [-s] [-u DATA_URL] [-D] [-v]

Build a dmrpp file for an HDF4 file. get_dmrpp_h4 -i h4_file_name. A dmrpp
file that uses the HDF4 file name will be generated.

optional arguments:

...
```

Lets build a DMR++ now, by explicitly using the container:

```
docker exec -it hyrax bash
```

starts the *bash* shell in the container, with the current directory as root (/)

```
[root@hyrax /]#
```

Change to the directory that is the root of the data (you'll see your HDF4 files in here):

```
 cd /usr/share/hyrax
```

You will see, roughly:

```
[root@hyrax /]# cd /usr/share/hyrax
[root@hyrax hyrax]# ls
3B42.19980101.00.7.HDF
3B42.19980101.03.7.HDF
3B42.19980101.06.7.HDF

...
```

In that directory, use the *get_dmrpp_h4* command to build a DMR++ document for one of the files:

```
[root@hyrax hyrax]# get_dmrpp_h4 -i 3B42.20130111.09.7.HDF -u
'file:///usr/share/hyrax/3B42.20130111.09.7.HDF'
```

Copy that pattern for whatever file you use. From the `/usr/share/hyrax` directory, you pass *get_dmrpp_h4* the name of the file (because it's local to the current directory) using the **-i** option. The **-u** option tells the command to embed the URL that follows it in the DMR++ . I've used a *file://* URL to the file */usr/share/hyrax/3B42.19980101.00.7.HDF*.

NOTE In the URL above, three slashes following the colon: two from the way a URL names a protocol and one because the pathname starts at the root directory.

Building the DMR++ and embedding a *file://* URL will enable testing the DMR++ .

4.1.2. Using the server to examine data returned by the DMR++

Lets look at how the *hyrax* service will treat that data file using the DMR++ . In a browser, go to http://localhost:8080/opendap/

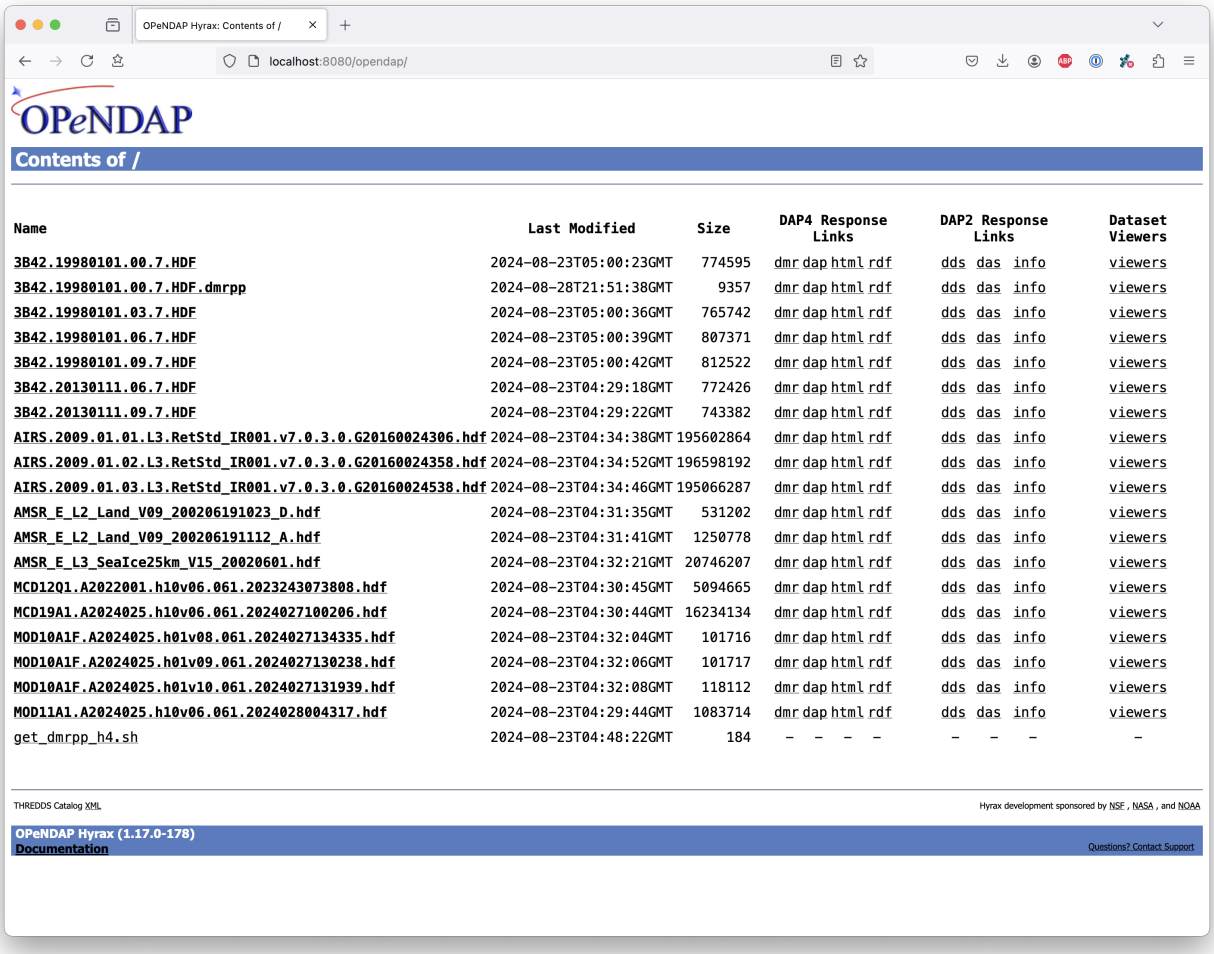

Figure 2. Hyrax Catalog view of all files available.

NOTE *The server caches data catalog information for 5 minutes (although this can be configured) so new items (e..g., DMR++ documents) may not show up right away. To*

> *force the display of a DMR++ that you just created, click on the source data file name and edit the URL so that the suffix* ***.dmr.html*** *is replaced by* ***.dmrpp/dmr*** *.*

Click on the your equivalent of the **3B42.20130111.09.7.HDF** link, subset, download and open in Panoply or the equivalent.

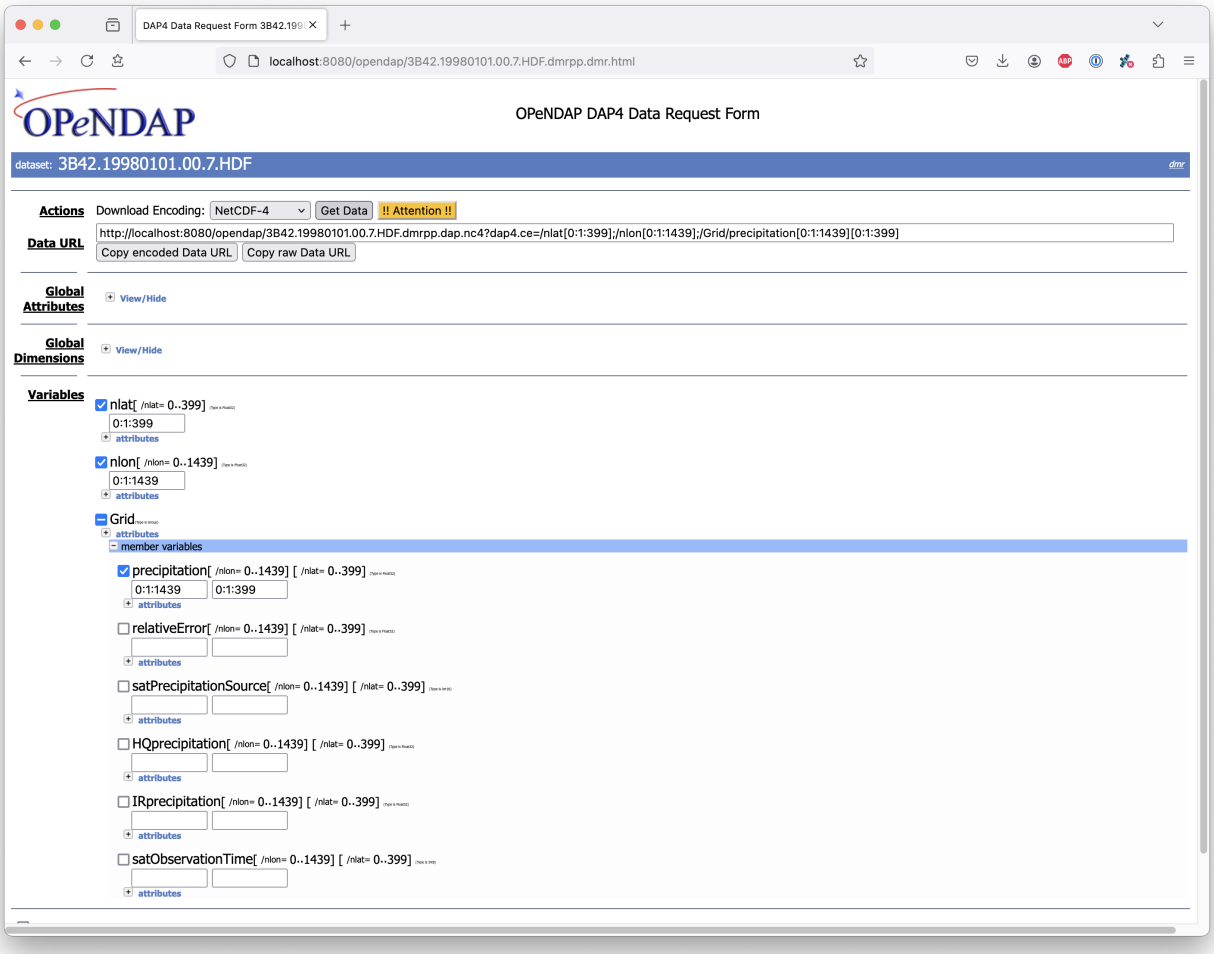

Figure 3. Page view of the DAP Data Request Form for subsetting the dataset.

You can run batch tests in lots of files by building many DMR documents and then asking the server for various responses (_nc4_, _dap_) from the + DMR + and the original file. Those could be compared using various schemes, although in its entirety that is beyond this section's scope, the command *getdap4* is also included in the container and could be used to compare *dap* responses from the data file and the DMR++ document.

Below is a comparison of the same underlying data, the left window shows the data returned using the DMR++ , the right shows the data read directly from the file using the server's builtin HDF4 reader.

[Data comparison] | */Data-comparison.png*

Figure 4. Comparison of responses from a DMR++ and the native file handler.

4.1.3. Simple shell command

Here is a simple shell command that you can run on the host computer that will eliminate most of

the above.

NOTE	"In the spirit of a recipe, I'll restate the earlier command for starting the docker container with the **get_dmrpp_h4** command and the **hyrax** server."

Start the container:

```
docker run -d -h hyrax -p 8080:8080 -v $HDF4_DIR:/usr/share/hyrax --name=hyrax
opendap/hyrax:snapshot
```

Check if it is running:

```
docker ps
```

The command, written for the Bourne Shell, is:

```
#!/bin/sh
#
# usage get_dmrpp_h4.sh <file>

data_root=/usr/share/hyrax

cat <<EOF | docker exec --interactive hyrax sh
cd $data_root
get_dmrpp_h4 -i $1 -u "file://$data_root/$1"
EOF
```

Copy that, save it in a file (I named the file *get_dmrpp_h4.sh*).

Run the command on the host (not the docker container) and in the directory with the HDF4 files (you don't have to do that, but sorting out the details is left as an exercise for the reader. Run the command like this:

```
 ./get_dmrpp_h4.sh AMSR_E_L3_SeaIce25km_V15_20020601.hdf
```

The DMR++ will appear when the command completes.

```
(hyrax500) hyrax_git/HDF4-dir % ls -l
total 1251240
-rw-r--r--@ 1 jimg  staff    1250778 Aug 22 22:31
AMSR_E_L2_Land_V09_200206191112_A.hdf
-rw-r--r--@ 1 jimg  staff   20746207 Aug 22 22:32
AMSR_E_L3_SeaIce25km_V15_20020601.hdf
-rw-r--r--  1 jimg  staff    3378674 Aug 28 17:37
```

```
AMSR_E_L3_SeaIce25km_V15_20020601.hdf.dmrpp
```

5. Building DMR++ files for HDF5/NetCDF4 with *get_dmrpp*

The application that builds the DMR++ files is a command line tool called *get_dmrpp*. It in turn utilizes other executables such as *build_dmrpp*, *reduce_mdf*, *merge_dmrpp* (which rely in turn on the *hdf5_handler* and the "hdf5" library), along with a number of UNIX shell commands.

All of these components are install with each recent version of the Hyrax Data Server

You can see the *get_dmrpp* usage statement with the command:

```
get_dmrpp -h
```

5.1. Using *get_dmrpp*

The way that *get_dmrpp* is invoked controls the way that the data are ultimately represented in the resulting DMR++ file(s).

The *get_dmrpp* application utilizes software from the Hyrax data server to produce the base DMR document which is used to construct the DMR++ file.

The Hyrax server has a long list of configuration options, several of which can substantially alter the the structural and semantic representation of the dataset as seen in the DMR++ files generated using these options.

5.2. Command line options

The command line switches provide a way to control the output of the tool. In addition to common options like verbose output or testing modes, the tool provides options to build extra (aka 'sidecar') data files that hold information needed for CF compliance if the original HDF5 data files lack that information (see the "missing data" section). In addition, it is often desirable to build DMR++ files before the source data files are uploaded to a cloud store like S3. In this case, the URL to the data may not be known when the DMR++ is built. We support this by using placeholder/template strings in the "dmr" and which can then be replaced with the URL at runtime, when the + DMR + file is evaluated. See the '-u' and '-p' options below.

5.2.1. Inputs

-b

The fully qualified path to the top level data directory. Data files read by *get_dmrpp* must be in the directory tree rooted at this location and their names expressed as a path relative to this location. The value may not be set to `/` , or `/etc`. The default value is `/tmp` if a value is not provided. All the data files to be processed must be in this directory or one of its subdirectories.

If *get_dmrpp* is being executed from same directory as the data then `-b `pwd` or `-b .` works as well.

-u

This option is used to specify the location of the binary data object. It's value must be an http, https, or file (file://) URL. This URL will be injected into the DMR++ when it is constructed. If option `-u` is not used; then the template string `OPeNDAP_DMRpp_DATA_ACCESS_URL` will be used and the DMR++ will substitute a value at runtime.

-c

The path to an alternate bes configuration file to use.

-s

The path to an optional addendum configuration file which will be appended to the default BES configuration. Much like the `site.conf` file works for the full server deployment it will be loaded last and the settings there-in will have an override effect on the default configuration.

5.2.2. Output

-o

The name of the file to create.

5.2.3. Verbose Output Modes

-h

Show help/usage page.

-v

verbose mode, prints the intermediate DMR.

-V

Very verbose mode, prints the DMR, the command and the configuration file used to build the DMR.

-D

Just print the DMR that will be used to build the DMR++ .

-X

Do not remove temporary files. May be used independently of the `-v` and/or `-V` options.

5.2.4. Tests

-T

Run ALL hyrax tests on the resulting DMR++ file and compare the responses the ones generated by the source hdf5 file.

-I

Run hyrax inventory tests on the resulting DMR++ file and compare the responses the ones

generated by the source hdf5 file.

-F

Run hyrax value probe tests on the resulting DMR++ file and compare the responses the ones generated by the source hdf5 file.

5.2.5. Missing Data Creation

-M

Build a 'sidecar' file that holds missing information needed for CF compliance (e.g., Latitude, Longitude and Time coordinate data).

-p

Provide the URL for the Missing data sidecar file. If this is not given (but -M is), then a template value is used in the DMR++ file and a real URL is substituted at runtime.

-r

The path to the file that contains missing variable information for sets of input data files that share common missing variables. The file will be created if it doesn't exist and the result may be used in subsequent invocations of *get_dmrpp* (using `-r`) to identify the missing variable file.

5.2.6. AWS Integration

The *get_dmrpp* application supports both S3 hosted granules as inputs, and uploading generated DMR++ files to an S3 bucket.

S3 Hosted granules are supported by default

When the `get_dmrpp` application sees that the name of the input file is an S3 URL it will check to see if the AWS CLI is configured and if so `get_dmrpp` will attempt retrieve the granule and make a DMR++ utilizing whatever other options have been chosen. **For example:**

```
get_dmrpp -b `pwd` s3://bucket_name/granule_object_id
```

-U

The `-U` command line parameter for `get_dmrpp` instructs `get_dmrpp` application to upload the generated DMR++ file to S3, but only when the following conditions are met:

- The name of the input file is an S3 URL.
- The `AWS CLI` has been configured with credentials that provide `r+w` permissions for the bucket referenced in the input file S3 URL.
- The `-U` option has been specified. If all three of the above are true then `get_dmrpp` will copy the retrieve the granule, create a DMR++ file from the granule, and copy the resulting DMR++ file (as defined by the `-o` option) to the source S3 bucket using the well known NGAP sidecar file naming convention: **s3://bucket_name/granule_object_id.dmrpp**. For example:

```
get_dmrpp -U -o foo -b `pwd` s3://bucket_name/granule_object_id
```

5.3. *hdf5_handler* Configuration

Because *get_dmrpp* uses the *hdf5_handler* software to build the DMR++ the software must inject the *hdf5_handler*'s configuration.

The default configuration is large, but any valued may be altered at runtime.

Here are some of the commonly manipulated configuration parameters with their default values:

```
H5.EnableCF=true
H5.EnableDMR64bitInt=true
H5.DefaultHandleDimension=true
H5.KeepVarLeadingUnderscore=false
H5.EnableCheckNameClashing=true
H5.EnableAddPathAttrs=true
H5.EnableDropLongString=true
H5.DisableStructMetaAttr=true
H5.EnableFillValueCheck=true
H5.CheckIgnoreObj=false
```

5.3.1. Note to DAACs with existing Hyrax deployments.

If your group is already serving data with Hyrax and the data representations that are generated by your Hyrax server are satisfactory, then a careful inspection of the localized configuration, typically held in `/etc/bes/site.conf`, will help you determine what configuration state you may need to inject into *get_dmrpp*.

5.4. The *H5.EnableCF* option

Of particular importance is the *H5.EnableCF* option, which instructs the *get_dmrpp* tool to produce Climate Forecast convention (CF) compatible output based on metadata found in the granule file being processed.

Changing the value of *H5.EnableCF* from **false** to **true** will have (at least) two significant effects.

It will:

- Cause *get_dmrpp* to attempt to make the dmr++ metadata CF compliant.
- Remove Group hierarchies (if any) in the underlying data granule by flattening the Group hierarchy into the variable names.

By default *get_dmrpp* the *H5.EnableCF* option is set to false:

```
H5.EnableCF = false
```

There is a much more comprehensive discussion of this key feature, and others, in the HDF5 Handler section of the Appendix in the Hyrax Data Server Installation and Configuration Guide.

5.5. Missing data, the CF conventions and *hdf5*

Many of the *hdf5* files produced by NASA and others do not contain the domain coordinate data (such as latitude, longitude, time, etc.) as a collection of explicit values. Instead information contained in the dataset metadata can used to reproduce these values.

In order for a dataset to be Climate Forecast (CF) compatible it must contain these domain coordinate data values.

The Hyrax *hdf5_handler* software, utilized by the *get_dmrpp* application, can create this data from the dataset metadata. The *get_dmrpp* application places these generated data in a “sidecar” file for deployment with the source *hdf5/netcdf-4* file.

6. Hyrax - Serving data using DMR++ files

There are three fundamental deployment scenarios for using DMR++ files to serve data with the Hyrax data server.

This can be simple categorized as follows: The DMR++ file(s) are XML files that contain a root `dap4:Dataset` element with a `dmrpp:href` attribute whose value is one of:

1. An http(s):// URL referencing to the underlying granule files via http.
2. A file:// URL that references the granule file on the local filesystem in a location that is inside the BES' data root tree.
3. The template string `OPeNDAP_DMRpp_DATA_ACCESS_URL`

Each will discussed in turn below.

NOTE	By default Hyrax will automatically associate files whose name ends with ".dmrpp" with the **DMR++** handler.

6.1. Using DMR++ with http(s) URLs

If the DMR++ files that you wish to serve contain `dmrpp:href` attributes whose values are http(s) URLs then there are 2+1 steps to serve the data:

1. Place the DMR++ files on the local disk inside of the directory tree identified by the `BES.Catalog.catalog.RootDirectory` in the BES configuration.
2. Ensure that the Hyrax `AllowedHosts` list is configured to allow Hyrax to access those target URLs. This can be accomplished by adding new regex entires to the `AllowedHosts` list in

`/etc/bes/site.conf`, creating that file as need be.

3. If the data URLs require authentication to access then you'll need to configure Hyrax for that too.

6.2. Using DMR++ with file URLs

Using DMR++ files with locally held files can be useful for verifying that DMR++ functionality is working without relying on network access that may have data rate limits, authenticated access configuration, or security access constraints. Additionally, in many cases the DMR++ access to the locally held data may be significantly faster than through the native `netcdf-4/hdf5` data handlers.

In order to use DMR++ files that contain file:// URLs: . Place the DMR++ files on the local disk inside of the directory tree identified by the `BES.Catalog.catalog.RootDirectory` in the BES configuration. . Ensure the the DMR++ files contain only file:// URLs that refer to data granule files inside of the directory tree identified by the `BES.Catalog.catalog.RootDirectory` in the BES configuration.

Note: For Hyrax, a correctly formatted file URL must start with the protocol `file://` followed by the full qualified path to the data granule, for example:

`/usr/share/hyrax/ghrsst/some_granule.h5`

so the the completed URL will have three slashes after the first colon:

`file:///usr/share/hyrax/ghrsst/some_granule.h5`

6.3. Using DMR++ with the template string (NASA).

Another way to serve DMR++ files with Hyrax is to build the DMR++ files **without** valid URLs but with a template string that is replaced at runtime. If no target URL is supplied to *get_drmpp* at the time that the DMR++ is generated the template string: `OPeNDAP_DMRpp_DATA_ACCESS_URL` will added to the file in place of the URL. The at runtime it can be replaced withe the correct value.

Currently the only implementation of this is Hyrax's NGAP service which, when deployed in the NASA NGAP cloud, will accept "restified path" URLs that are defined as having a URL path component with two mandatory and one optional parameters:

```
MANDATORY: "/collections/UMM-C:{concept-id}"
OPTIONAL:  "/UMM-C:{ShortName} '.' UMM-C:{Version}"
MANDATORY: "/granules/UMM-G:{GranuleUR}"
```

Example: https://opendap.earthdata.nasa.gov/collections/C1443727145-LAADS/MOD08_D3.v6.1/granules/MOD08_D3.A2020308.061.2020309092644.hdf.nc

When encountering this type of URL Hyrax will decompose it and use the content to formulate a query to the NASA CMR in order to retrieve the data access URL for the granule and for the DMR++ file. It then retrieves the DMR++ file and injects the data URL so that data access can proceed as described above.

More on the Restified Path can be found here (NOTE: You need the right permissions access the previous URL).

7. Recipe: Building and testing DMR++ files

There are two recipes shown here, one using Hyrax docker containers and a second using the container that is part of the EOSDIS Cumulous task.

Prerequisites:

- Docker daemon running on a system that also supports a shell (the examples use bash in this section).

7.1. Recipe: Building DMR++ files using a Hyrax docker container

1. Acquire representative granule files for the collection you wish to import. Put them on the system that is running the Docker daemon. For this recipe we will assume that these files have been placed in the directory:

```
/tmp/dmrpp
```

2. Get the most up to date Hyrax docker image:

```
docker pull opendap/hyrax:snapshot
```

3. Start the docker container, mounting your data directory on to the docker image at `/usr/share/hyrax`:

```
docker run -d -h hyrax -p 8080:8080 --volume /tmp/dmrpp:/usr/share/hyrax
--name=hyrax opendap/hyrax:snapshot
```

4. Get a first view of your data using `get_dmrpp` with it's default configuration.
   a. If you want you can build a DMR++ for an example "input_file" using a docker exec command:

```
docker exec -it hyrax get_dmrpp -b /usr/share/hyrax -o
/usr/share/hyrax/input_file.dmrpp -u "file:///usr/share/hyrax/input_file"
"input_file"
```

   b. Or if you want more scripting flexibility you can login to the docker container to do the same:
      i. Login to the docker container:

```
docker exec -it hyrax /bin/bash
```

ii. Change working dir to data dir:

```
cd /usr/share/hyrax
```

iii. Set the data directory to the current one (`-b $(pwd)`) and set the data URL (`-u`) to the fully qualified path to the input file.

```
get_dmrpp -b $(pwd) -o foo.dmrpp -u "file://"$(pwd)"/your_test_file"
"your_test_file"
```

NOTE Now that you have made a dmr++ file, use the running Hyrax server to view and test it by pointing your browser at: http://localhost:8080/opendap/

5. You can also batch process all of your test granules, if you want to go that route. This script assumes your ingestable data files end with '.h5'.

NOTE The resulting **DMR++** files should contain the correct file:// URLs and be correctly located so that they may be tested with the Hyrax service running in the docker instance.

```
#!/bin/bash
# This script will write each output file as a sidecar file into
# the same directory as its associated input granule data file.

# The target directory to search for data files
target_dir=/usr/share/hyrax
echo "target_dir: ${target_dir}";

# Search the target_dir for names matching the regex \*.h5
for infile in `find "${target_dir}" -name \*.h5`
do
    echo " Processing: ${infile}"

    infile_base=`basename "${infile}"`
    echo "infile_base: ${infile_base}"

    bes_dir=`dirname "${infile}"`
    echo "    bes_dir: ${bes_dir}"

    outfile="${infile}.dmrpp"
    echo "     Output: ${outfile}"

    get_dmrpp -b "${bes_dir}" -o "${outfile}" -u "file://${infile}" "${infile_base}"
```

```
done
```

TIP	Remember that you can use the Hyrax server that is running in the docker container to view and test the DMR++ files you just created by pointing your browser at: http://localhost:8080/opendap/

7.2. Testing and qualifying DMR++ files

In the previous section/step we created some initial DMR++ files using the default configuration. It is crucial to make sure that they provide the representation of the data that you and your users are expecting, and that they will work correctly with the Hyrax server. (See the following sections for details). If the generated DMR++ files do not match expectations then the default configuration of the `get_dmrpp` may need to be amended using the `-s` parameter. If the data are currently being served by your DAAC's on-prem team this is where understanding exactly what the localizations made to the configurations of the on-prem Hyrax instances deployed for the collection is important. These localization will probably need to be injected into `get_drmpp` in order to produce the correct data representation in the DMR++ files.

7.3. Flattening Groups

By default `get_dmrpp` will preserve and show group hierarchies. If this is not desired, say for CF-1.0 compatibility, then you can change this by creating a small amendment to `get_dmrpp's default configuration.

First create the amending configuration file:

```
echo "H5.EnableCF=true" > site.conf
```

Then, change the invocation of `get_dmrpp` in the above example by adding the `-s` switch:

```
get_dmrpp -s site.conf -b `pwd` -o "${dmrpp_file}" -u "file://"`pwd`"/${file}"
"${file}"
```

And re-run the DMR++ production as shown above.

7.4. DAP representations

We have test and assurance procedures for DAP4 and DAP2 protocols below. Both are important. For legacy datasets the DAP2 request API is widely used by an existing client base and should continue to be supported. Since DAP4 subsumes DAP2 (but with somewhat different API semantics) It should be checked for legacy datasets as well. For more modern datasets that content DAP4 types such as Int64 that are not part of the DAP2 specification or implementations we will need to relying eliding the instances of unmapped types, or return an error when this is encountered.

```
# Test Constants:
GRANULE_FILE="some_name.h5"
# Granule URL
gf_url="http://localhost:8080/opendap/${GRANULE_FILE}"
```

7.4.1. Inspect the DMR++ files

Do the DMR++ files have the expected `dmrpp:href` URL(s)?

```
head -2 ${GRANULE_FILE}.dmrpp
```

7.4.2. Check DAP4 DMR Response

Inspect `${gf_url}.dmrpp.dmr`

1. Get the document, save as `foo.dmr`:

   ```
   curl -L -o foo.dmr "${gf_url}.dmr"
   ```

2. Is each variable's data type correct and as expected?
3. Are the associated dimensions correct?

7.4.3. DAP4 Check binary data response

For a particular granule GRANULE_FILE and a particular variable VARIABLE_NAME (Where VARIABLE_NAME is a full qualified DAP4 name):

```
curl -L -o dap4_subset_file "${gf_url}.dap?dap4.ce=VARIABLE_NAME"
curl -L -o dap4_subset_dmrpp "${gf_url}.dmrpp.dap?dap4.ce=VARIABLE_NAME"
cmp dap4_subset_file dap4_subset_dmrpp
```

7.4.4. DAP4 UI test

View and exercise the DAP4 Data Request Form `{gf_url}.dmr.html`

7.4.5. DAP2 Check DDS Response

1. Inspect `${gf_url}.dds`
   a. Is each variable's data type correct and as expected?
   b. Are the associated dimensions correct?
2. Compare DMR++ DDS with granule file DDS. :: For a particular granule GRANULE_FILE and a particular variable VARIABLE_NAME (Where VARIABLE_NAME is a DAP2 name):

```
curl -L -o dap2_dds_file "${gf_url}.dds"
curl -L -o dap2_dds_dmrpp "${gf_url}.dds"
cmp dap2_dds_file dap2_dds_dmrpp
```

7.4.6. DAP2 Check binary data response

For a particular granule GRANULE_FILE and a particular variable VARIABLE_NAME (Where VARIABLE_NAME is a DAP2 name):

```
curl -L -o dap2_subset_file "${gf_url}.dods?VARIABLE_NAME"
curl -L -o dap2_subset_dmrpp "${gf_url}.dmrpp.dods?VARIABLE_NAME"
cmp dap2_subset_file dap2_subset_dmrpp
```

NOTE One might consider doing this with two or more variables.

7.4.7. DAP2 UI Test

1. View and exercise the DAP2 Data Request Form located here: `{gf_url}.html`.
2. Try it in Panoply!
   a. Open Panoply.
   b. From the File menu select **Open Remote Dataset**...
   c. Paste the `{gf_url}.html` into the resulting dialog box.